4/10/2026

Phase 1

Project Planning and Management

Kobo Ts'epo Ramosoeu
202322608

1. Project Planning and Management

1.1 Feasibility Study

1.1.1 Technical feasibility

The required technology is mature and suits the constraints. SQLite is an embedded, serverless database, so, it works offline on the school's old computers without a database server, which answers the IT officer's main concern. 800 students is a small data volume for SQLite when the schema is indexed. One limit matters for the deployment design: SQLite's own guidance recommends a client/server database when many programs access one database over a network [11]. The brief allows a local network, so I will specify at Gate 2 that the database file is accessed by a single application instance on one machine, and treat multi-machine access as a documented limitation rather than a hidden defect. Salted password hashing can use libsodium, which provides the Argon2id algorithm [12], and Google Test provides the unit-test framework [13]. My main technical uncertainty is integrating CMake, SQLite, libsodium, Google Test and a CI service in one build. I treat this as a risk (R5) and remove it early with a technical spike in week 1. **Rating: feasible with conditions** (single-machine database access; spike completed in week 1).

1.1.2 Operational feasibility

The system must fit how the school already works: lecturers mark, HoDs moderate, and students see marks only after release. The Principal states that teachers are not IT experts, so the user interface must be simple. A role-based console menu with few options per role, input validation with clear messages, and a user manual per role (Phase 7) address this. The offline requirement is met by the architecture and the audit log supports accountability for mark changes. The main operational risk is that stakeholders disagree about when marks are released. This is a requirements conflict and not a feasibility blocker, and I will resolve it in Phase 2 with a documented rule. **Rating: feasible.**

1.1.3 Economic feasibility

The project has no license or hardware cost. Every tool is free or open source: SQLite is in the public domain [11], libsodium is released under the ISC license [12], Google Test under the BSD 3-clause license [13], and the school's existing computers are sufficient because the application is a lightweight console program. The real cost is my time. For the school, the one-off costs are installation and one short training session per role (assumed 1 to 2 hours each), and the benefits are fewer marking and moderation errors, consistent late-penalty calculation and a complete audit trail of mark changes. I have not put monetary values on these benefits because the brief gives no figures, and inventing them would not add credibility. **Rating:**
feasible.

1.1.4 Legal/ethical feasibility

The system stores personal information about learners, most of whom are under 18. Lesotho's Data Protection Act, 2011 (published as Act No. 5 of 2012) regulates the processing of personal information and defines a child as a person under 18 [10]. Reports indicate that the Data Protection Commission the Act creates has not yet been appointed, so enforcement is currently weak, but I treat the Act's principles as binding design requirements all the same: collect only the data needed, restrict access by role, protect stored data (salted password hashes, no plain-text credentials) and keep an audit trail. I will discuss this in the professional, legal and ethical section of the final report and I am not giving legal advice here. All tools used have licenses that permit this use without copyleft obligations on my code (cppcheck, used only as an analysis tool, is not distributed with the product). As this is assessed individual work, all sources must be cited and the code must be my own. **Rating: feasible with conditions** (synthetic test data only; data-protection principles built into the design).

1.1.5 Schedule feasibility

The assignment provides the following high-level schedule:

Period	Activity
Week 1	Planning
Weeks 1–2	Requirements

Weeks 2–3	System design
Weeks 4–5	Implementation
Weeks 5–6	Verification & Validation
Week 6	Deployment
Week 7	Maintenance/completion

The seven-week period is achievable if the project follows the prescribed phase gates and dependencies and avoids beginning implementation before requirements and design have been sufficiently completed. **Rating: feasible with conditions (with disciplined project management).**

1.1.6 Summary and Recommendation

Dimension	Main finding	Rating
Technical	SQLite, C++17, libsodium and Google Test are suitable; toolchain integration is the uncertainty; network access to SQLite is a limit	Feasible with conditions
Operational	Role-based console UI and manuals suit non-expert staff; offline operation is met	Feasible
Economic	No license or hardware cost; main cost is my time	Feasible
Legal	Learners are mostly	Feasible with conditions

	children; data-protection principles apply; licenses are compatible	
Schedule	The seven-week period is achievable if the project follows the prescribed phase gates and dependencies	Feasible with conditions

Recommendation: proceed with the project, subject to five conditions:

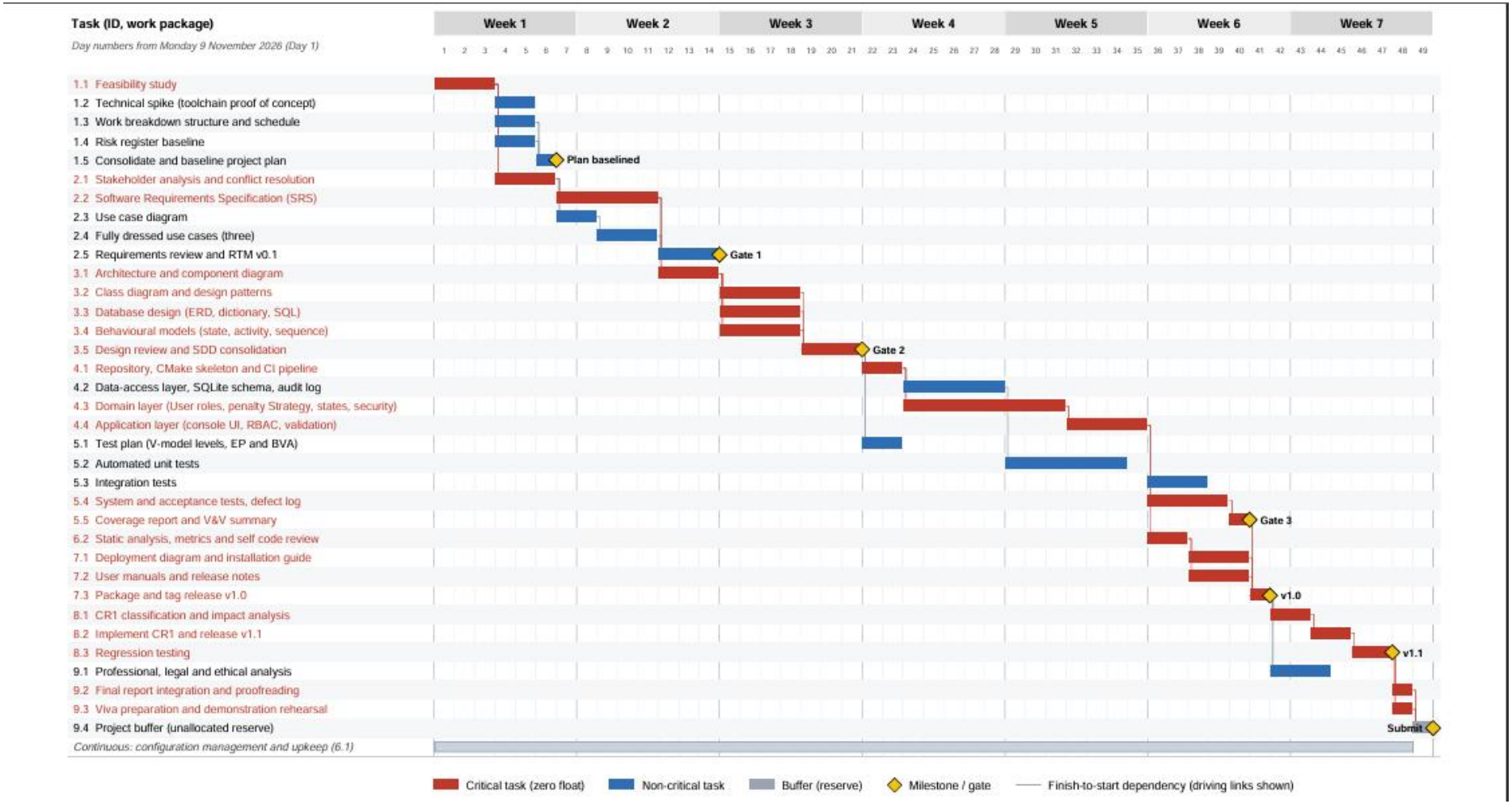
- **C1** Freeze scope at Gate 1 using MoSCoW; implement only Must requirements until they all pass their tests.
- **C2** Complete the technical spike (task 1.2) by day 5 and use it to confirm the build, test and CI configuration.
- **C3** Use synthetic data only and build the data-protection principles into the design.
- **C4** Specify single-machine access to the SQLite file at Gate 2 and state the limitation in the documentation.
- **C5** Track planned against actual hours weekly and apply the scope fallback list if the cumulative slip exceeds 10%.

1.2 Work Breakdown Structure (WBS)

Moderated LMS project								
1 Project planning	**2 Requirements engineering**	**3 System design**	**4 Implementation**	**5 Verification and validation**	**6 Configuration management and QA**	**7 Deployment**	**8 Maintenance**	**9 Completion**
1.1 Feasibility study	2.1 Stakeholder analysis and conflict resolution	3.1 Architecture and component diagram	4.1 Repository, CMake skeleton and CI pipeline	5.1 Test plan	6.1 CM, RTM upkeep and weekly reviews	7.1 Deployment diagram and installation guide	8.1 CR1 classification and impact analysis	9.1 Professional, legal and ethical analysis
1.2 Work breakdown structure and schedule	2.2 Software Requirements Specification	3.2 Class diagram and design patterns	4.2 Data-access layer, SQLite schema, audit log	5.2 Automated unit tests	6.2 Static analysis, metrics and self-code review	7.2 User manuals and release notes	8.2 Implement CR1 and release v1.1	9.2 Final report integration and proofreading
1.3 Risk register baseline	2.3 Use case diagram	3.3 Database design	4.3 Domain layer	5.3 Integration tests		7.3 Package and tag release v1.0	8.3 Regression testing	9.3 Viva preparation and demonstration rehearsal
	2.4 Fully dressed use cases	3.4 Behavioral models	4.4 Application layer	5.4 System and acceptance tests, defect log				

	2.5 Requirements review and RTM v0.1	3.5 Design review and SDD consolidation		5.5 Coverage report and V&V summary				

Gnatt Chart


Task (ID, work package)
Day numbers from Monday 9 November 2026 (Day 1)
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
1.1 Feasibility study
1.2 Technical spike (toolchain proof of concept)
1.3 Work breakdown structure and schedule
1.4 Risk register baseline
1.5 Consolidate and baseline project plan
Plan baselined
2.1 Stakeholder analysis and conflict resolution
2.2 Software Requirements Specification (SRS)
2.3 Use case diagram
2.4 Fully dressed use cases (three)
2.5 Requirements review and RTM v0.1
Gate 1
3.1 Architecture and component diagram
3.2 Class diagram and design patterns
3.3 Database design (ERD, dictionary, SQL)
3.4 Behavioural models (state, activity, sequence)
3.5 Design review and SDD consolidation
Gate 2
4.1 Repository, CMake skeleton and CI pipeline
4.2 Data-access layer, SQLite schema, audit log
4.3 Domain layer (User roles, penalty Strategy, states, security)
4.4 Application layer (console UI, RBAC, validation)
5.1 Test plan (V-model levels, EP and BVA)
5.2 Automated unit tests
5.3 Integration tests
5.4 System and acceptance tests, defect log
5.5 Coverage report and V&V summary
Gate 3
6.2 Static analysis, metrics and self code review
7.1 Deployment diagram and installation guide
7.2 User manuals and release notes
7.3 Package and tag release v1.0
v1.0
8.1 CR1 classification and impact analysis
8.2 Implement CR1 and release v1.1
8.3 Regression testing
v1.1
9.1 Professional, legal and ethical analysis
9.2 Final report integration and proofreading
9.3 Viva preparation and demonstration rehearsal
9.4 Project buffer (unallocated reserve)
Submit
Continuous: configuration management and upkeep (6.1)
Critical task (zero float)
Non-critical task
Buffer (reserve)
Milestone / gate
Finish-to-start dependency (driving links shown)

Gantt chart with finish-to-start dependencies, milestones and critical path. Critical tasks are shown in red: 1.1, 2.1, 2.2, 3.1, 3.2/3.3/3.4, 3.5, 4.1, 4.3, 4.4, then two equal branches (5.4, 5.5 and 6.2, 7.1/7.2) rejoining at 7.3, then 8.1, 8.2, 8.3 and 9.2/9.3, followed by the 1-day buffer (9.4) that ends the project on day 49.

1.3 Risk Register

ID	Risk	Category	L	I	E	Mitigation	Owner
R1	Conflicting stakeholder requirements	Project	3	4	12	Conduct stakeholder analysis; document conflicts; agree explicit priorities before Gate 1.	Project Manager
R2	Requirements/design take longer than planned	Project	3	5	15	Use WBS and schedule; conduct progress reviews; prioritize critical deliverables; enforce phase gates.	Project Manager
R3	SQLite/database integration problems	Technical	3	4	12	Finalize ERD/schema before implementation; use a data access layer; perform early database tests.	Developer
R4	Authentication/authorization weaknesses	Technical	2	5	10	Use salted password hashes, role-based access, parameterized queries, security tests and code review.	Developer
R5	Unfamiliar toolchain integration (CMake, SQLite, libsodium, Google Test, GitHub Actions). Result: delays at the start of implementation.	Technical	3	3	9	Technical spike (task 1.2) in week 1 that builds, tests and runs in CI; keep dependencies to the minimum required by the brief; record working configurations in the build guide.	Developer
R6	Gate 2 delayed or not passed	Project	2	5	10	Track design deliverables; conduct early reviews; resolve traceability inconsistencies before the gate.	Project Manager
R7	Mark audit trail fails	Technical	2	5	10	Design audit logging explicitly; test every mark-	Developer

						change scenario; verify who, when and why.	